LAPORAN PRAKTIKUM
INTERNET OF THINGS II



Disusun Oleh:

Shavina Naura Putri

25782055

Dosen Pengampu:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, S.Kom

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

Praktikum 2

7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

sketch_oct19a.ino

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// 1. Kredensial Wi-Fi (Pastikan Laptop dan ESP8266 berada di Jaringan yang Sama!)
const char* ssid     = "blablabla";
const char* password = "yayayaya";

// 2. Konfigurasi Broker MQTT Lokal
// Ganti dengan Alamat IPv4 Laptop Anda yang didapat dari perintah 'ipconfig'!
const char* mqttServer = "10.71.190.66";
const int   mqttPort   = 1883;

// 3. Konfigurasi Identitas Simpul & Topik
const char* clientId      = "ESP8266_Local_Node01";
const char* topicPubTele  = "kampus/labiot/telemetri";
const char* topicSubCmd   = "kampus/labiot/kendali";

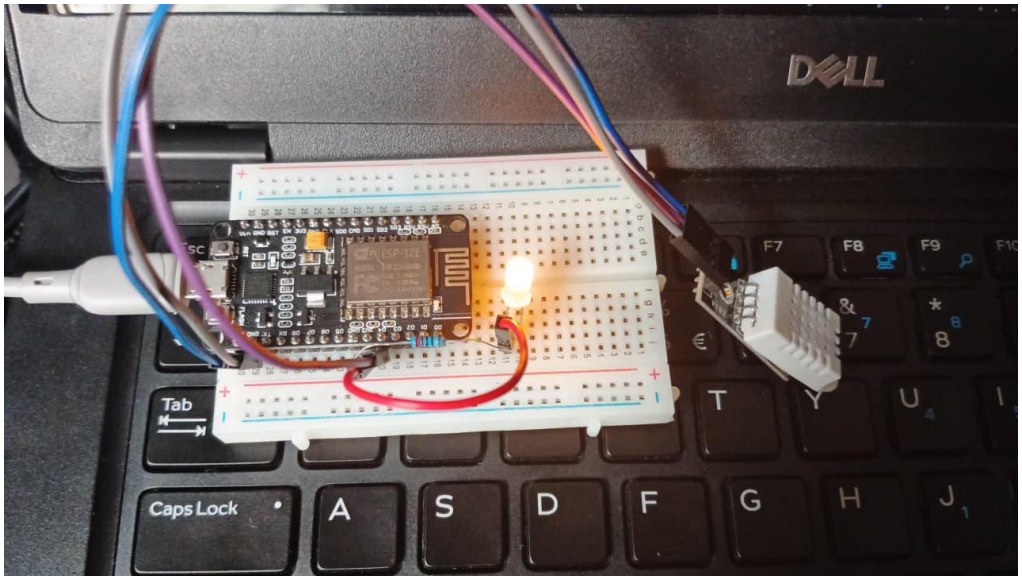
// 4. Alokasi Pin Perangkat Keras
const byte dhtPin = 2; // Pin D4 (GPIO 2)
const byte ledPin = 5; // Pin D1 (GPIO 5)

#define DHTTYPE DHT22 // Ganti DHT22 jika modul sensor berwarna putih

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
```

Output Serial Monitor

```
15:45:31.319 -> .............
15:45:37.629 -> [Wi-Fi] Berhasil Terhubung!
15:45:37.629 -> [Wi-Fi] IP NodeMCU: 10.71.190.145
15:45:37.629 -> [MQTT] Menghubungkan ke Broker Lokal (10.71.190.66)... Terhubung!
15:45:37.925 -> [MQTT] Berlangganan ke topik: kampus/labiot/kendali
15:45:37.925 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":35.10,"kelembapan":61.70,"status_led":0}
15:45:41.932 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":35.10,"kelembapan":61.90,"status_led":0}
15:45:45.951 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":35.10,"kelembapan":62.10,"status_led":0}
15:45:49.961 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":35.10,"kelembapan":60.20,"status_led":0}
15:45:53.920 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":35.00,"kelembapan":59.20,"status_led":0}
15:45:57.930 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":35.00,"kelembapan":59.00,"status_led":0}
15:46:01.923 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":35.00,"kelembapan":59.90,"status_led":0}
15:46:05.930 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":35.00,"kelembapan":61.00,"status_led":0}
15:46:09.919 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":35.00,"kelembapan":61.60,"status_led":0}
15:46:13.958 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":35.00,"kelembapan":60.50,"status_led":0}
```



Pada tahap ini, dilakukan proses migrasi pengiriman data telemetri dari mikrokontroler NodeMCU ESP8266 yang sebelumnya memanfaatkan *public broker* agar beralih menggunakan infrastruktur server lokal (*Mosquitto Broker*) yang berjalan di dalam kontainer Docker pada laptop.
Adapun poin-poin penjelasan penting yang perlu dicantumkan dalam laporan adalah:

1. **Penyesuaian Jaringan & Konfigurasi IP:** Alamat server MQTT pada program Arduino disesuaikan dengan alamat IPv4 lokal laptop pengembang (misalnya 192.168.1.15) yang didapat melalui perintah ipconfig. Hal ini memastikan NodeMCU di lapangan dapat berkomunikasi langsung dengan *Edge Gateway* di dalam jaringan area lokal (LAN) yang sama tanpa harus keluar ke internet publik.
2. **Pengaturan Keamanan Firewall (*Inbound Rules*):** Dilakukan penambahan aturan pada *Windows Defender Firewall* untuk membuka port TCP 1883. Langkah ini bersifat krusial agar sistem operasi Windows tidak memblokir paket data masuk dari mikrokontroler fisik luar menuju komputer host.
3. **Mekanisme Kode Program ESP8266:**
    - **Inisialisasi Sensor & Wi-Fi:** Program membaca suhu dan kelembapan secara berkala menggunakan sensor DHT11/DHT22 pada pin D4 (GPIO 2) serta mengelola status aktuator LED pada pin D1 (GPIO 5).
    - **Penyusunan Paket JSON:** Data sensor dikemas secara terstruktur menggunakan pustaka ArduinoJson ke dalam format objek JSON ({"node_id": "...", "suhu": ..., "kelembapan": ..., "status_led": ...}) sebelum dipublikasikan ke topik kampus/labiot/telemetri.
    - **Komunikasi Dua Arah (Callback):** Fungsi onMessageCallback disiapkan untuk mendengarkan instruksi kendali balik dari server lokal (melalui topik kampus/labiot/kendali), sehingga mikrokontroler dapat langsung mengubah status nyala/padam LED secara otonom berdasarkan perintah yang diterima.